

Agentic Coding

Sven Helmer

Kathrin Paschen

Chapter 1

Introduction

Motivation

- There's a lot of hype around vibe coding at the moment

“No more humans coding in 5 years.”

Emad Mostaque

“AI will write most of the code soon and perform better than top coders.”

Mark Zuckerberg

“It's our job to create computing technology such that nobody has to program.”

Jensen Huang

Motivation (2)

- This is not the first hype we have seen:

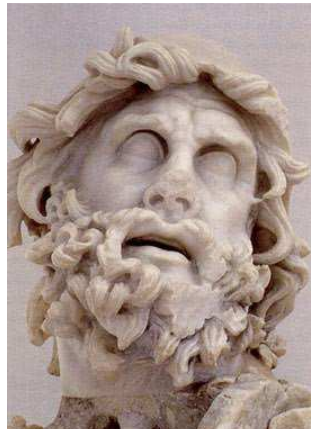


Things are Definitely Changing

- In the foreseeable future, AI will not replace developers
 - A developer using AI will replace one who doesn't
 - and developers (already) work differently
- Currently, AI boosts the skills of experts
- It will not save you if you have no clue what you're doing

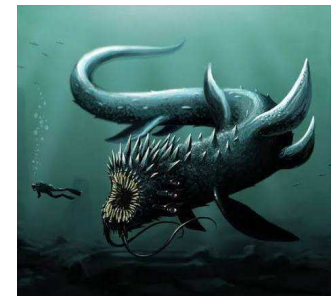
Basic Terminology

- Coding with AI covers a whole spectrum
 - Vibe coding: prompt and hope for the best
 - Agentic coding: a systematic approach that still utilizes software development principles



vibe coding

agentic coding



Vibe Coding

- Vibe coding typically looks like the following:
 - Write a prompt, e.g. “Build an app that does X.”
 - LLM generates some code
 - User tries it out, notices some things that do not work
 - User writes another prompt requesting changes
 - LLM changes code
 - Wash, rinse, repeat

Agentic Coding

- Gives an LLM specific software development tasks, such as:
 - Specify goal
 - Devise a plan: break up overall task into subtasks
 - Generate code
 - Validate code (for example by running tests)
 - Revise plan or implementation
 - Repeat until satisfied

Vibe-Coding vs. Agentic Coding

- Vibe coding isn't always bad, and agentic coding isn't always the right approach
- Vibe coding:
 - makes sense for rapid prototyping
 - is for quickly trying out things
- Agentic coding
 - is a thorough process
 - creates reliable code

What Can Go Wrong?

- AI gets stuck in infinite loops
- Deliver incorrect solutions confidently
- Cheat: optimize for validation such as benchmarks
- Implements what you said, not what you meant
 - General problem, existed before AI

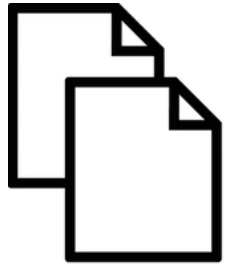
Summary

- Then: translate ideas into code (programming)
- Now: translate ideas into prompts (vibe-coding)
- Then: (systematically) applying principles and methods to create reliable software (software engineering)
- Now: give an AI (software-engineering) goals and constraints to create software (agentic coding)
- Important question: how much of the software-engineering process can be delegated?

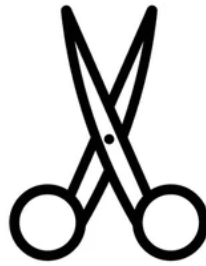
Chapter 2

The Environment

Basics



Copy



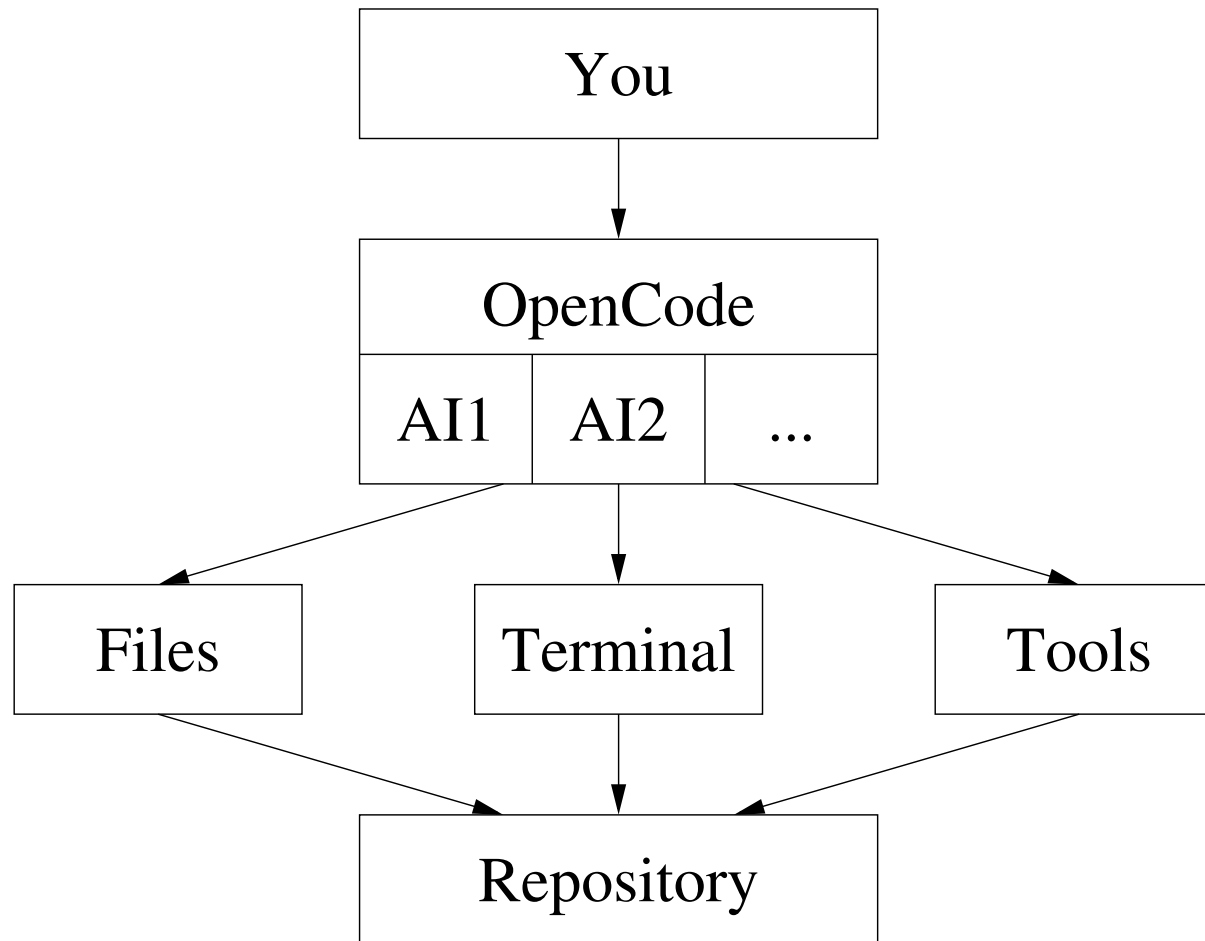
Cut



Paste

- What we don't want to do:
 - Prompting LLM on a web interface
 - Then copy/pasting code into file
- We use a more sophisticated environment

Overview



OpenCode

- OpenCode is an open-source coding tool
- Connects LLMs to development tools
- Model-agnostic: can interact with (almost) every model you want
- By default, code is created and stored locally

The logo for OpenCode, featuring the word "open" in a light gray, lowercase, sans-serif font, followed by "code" in a bold, black, lowercase, sans-serif font. The letters are slightly shadowed, giving a 3D effect.

Interaction with Files

- OpenCode is not an editor:
 - You can use any editor you want (VS Code, Emacs, IntelliJ) to look at the files
 - OpenCode executes commands in a terminal
- With these commands it can manipulate files and use other tools
 - This has direct implications for security

Repositories

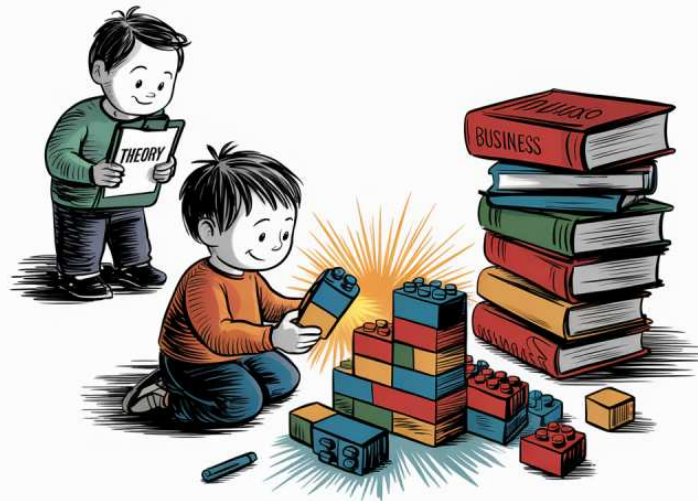
- A repository is where a version control system (VCS) stores its data
- A VCS tracks and manages changes to files
 - Heavily used in managing source code
- Allows to revert to a previous state
- Facilitates collaboration

Model Context Protocol (MCP)

- We just mention it for sake of completeness
- It is an open standard specifying how LLMs share data with external tools
- MCP offers a standardized interface for
 - reading files
 - executing functions
 - handling prompts

Summary

- This chapter is kept short on purpose
- We show how to set up and use this environment in hands-on exercises



THEORY TELLS. PRACTICE TEACHES.

Chapter 3

A Mental Model of LLMs and Agentic Coding

Introduction

- The aim is not to give a detailed technical description of LLMs (and the agents utilizing them)
 - You don't need to build an LLM or agent,
 - but to understand how they react to your input
- In a nutshell, an LLM is autocomplete on steroids
 - It doesn't look up an answer
 - It generates its response incrementally, depending on the context and the data it was trained on

Autocomplete vs. LLM

- If you type a message on your phone, it tries to predict how the message continues
 - Uses a dictionary and probabilities for continuations
- An LLM works at a different scale
 - Instead of a dictionary, uses a neural network with billions or trillions of parameters
 - Utilizes a much larger context (thousands of words)
 - Depending on the setting, it may not always return the most probable answer

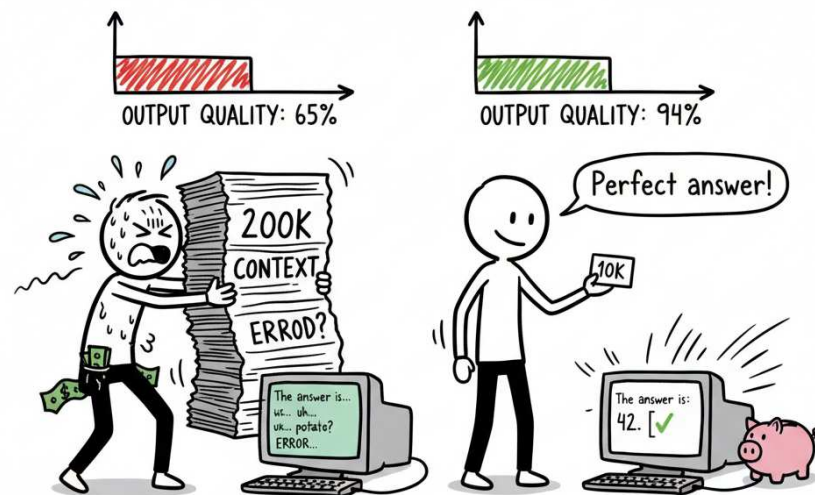
Tokens

- An LLM doesn't work with words, uses tokens instead
- We only give a brief introduction to tokens
- What do tokens look like?
 - (Short) words, e.g. *house*
 - Parts of words, e.g. *un con stitu tion al*
 - Punctuation, whitespace, emojis
- Tokens are used
 - to avoid vocabulary explosion
 - identify new words
 - describe concepts independent of language

Generating an Answer

- Context: current prompt and previous input/output
- An LLM then generates a plausible continuation:
context → next token → next token → next token
- A generated token is added to the context
- The *context window* plays a crucial role

THE CONTEXT WINDOW PARADOX



Context Window

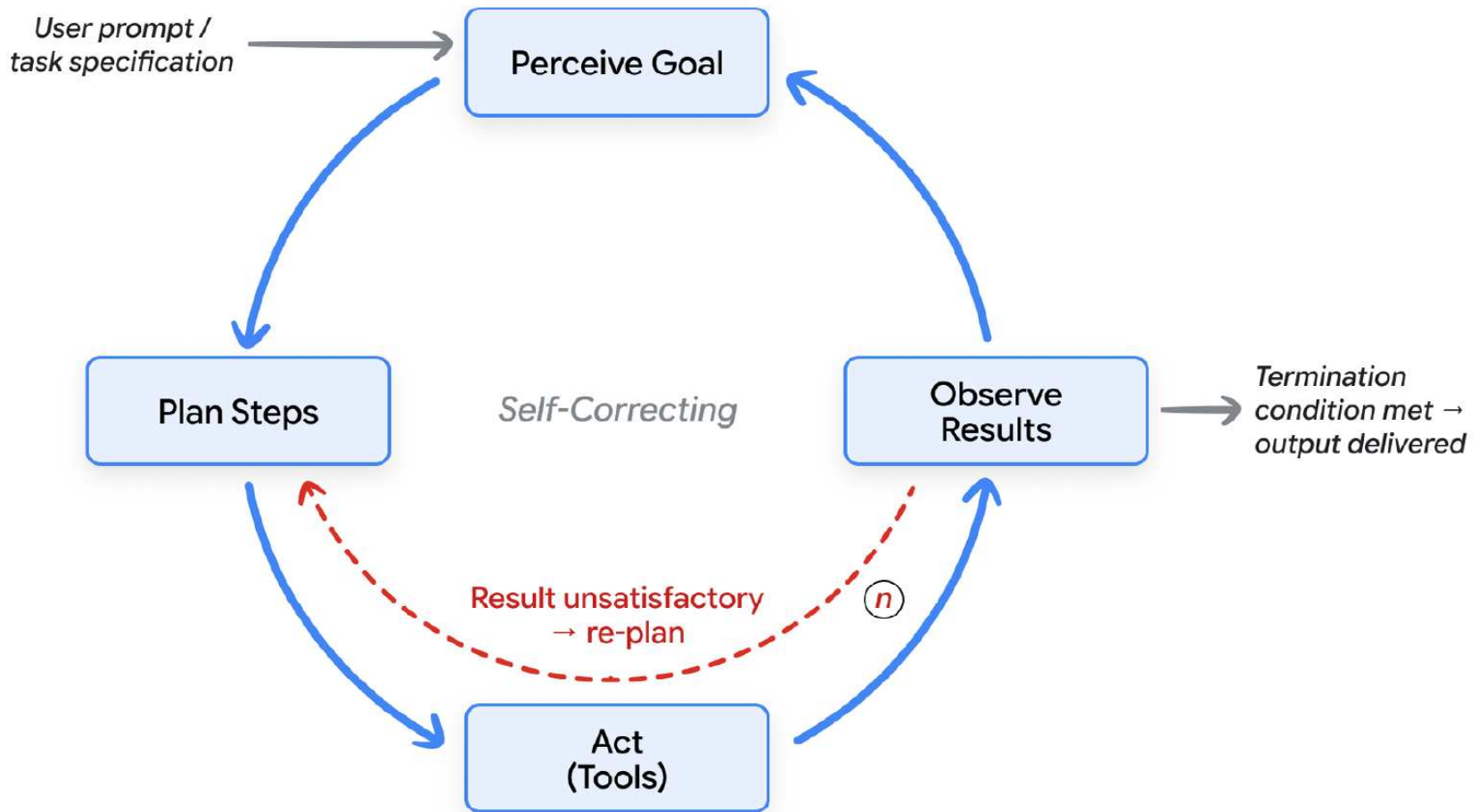
- Process is driven by the content of the context window
 - This window is not infinitely large
 - Higher recall at edges (beginning/end)
- Some similarity to our brain's working memory
 - Can hold seven chunks of information (+/-2)
 - Serial position effect (easier to remember start/end)
- Takeaway: if we want good results, we have to provide the right context

From Chatbot to Agent

- We don't want to use the LLM in the following way:
prompt → answer
- First of all, there's an issue with reliability
 - LLMs can be confidently incorrect
- Given a prompt (and all their learned patterns), an LLM gives plausible answers
 - LLMs do not know what the outside world looks like
- We want evidence for correctness
 - Idea: give LLMs access to tools for checking

From Chatbot to Agent (2)

- We want to follow a loop:



From Chatbot to Agent (3)

- In the context of coding, we provide tools such as
 - Requirement/specification documents
 - Code files
 - Tests
 - Current repository and history
 - External documentation
- In agentic coding, the LLM itself is not the agent
 - The agent is the system driving the above loop and using the LLM

Summary

- We covered a basic mental model of LLMs and agents
- Helps you in answering questions like:
 - Why did the LLM/agent just do this?
 - How can I make it behave differently?
- We briefly explained how LLMs/agents interact with the outside world

Chapter 4

From Problem to Specification

Introduction



- LLMs/agents don't magically know what you mean
- An agent can only implement
 - what you have specified
 - or what it can safely infer
- If your specification has gaps, the agent will fill them
- Results can range from *hilarious* to *dangerous*

A Cautionary Tale

- Assume you want to analyze some experimental data
- You enter the following as a prompt:
“Write a program that analyzes experimental data.”
- What do you expect the agent to produce?

THAT'S THE UGLIEST MESS
OF CODE I'VE EVER SEEN.
WHAT ON EARTH ARE YOU
WORKING ON?



A Cautionary Tale (2)

- To generate sensible code, an agent needs to know:
 - What kind of experiment?
 - What do we mean by “analyze”?
 - Which questions do we want answered?
 - What does the data look like?
 - What output do we want?
 - Who will use the result?
 - How do we check correctness?
- We need to define a lot of different aspects

Covering Different Aspects

- **Problem:** what is the general goal?
- **Requirements:** what should the program do?
- **Specification:** how should it do it?
- **Constraints:** what must/mustn't it do?
- **Acceptance criteria:** how do we check it's correct?
- **Examples:** what do concrete use cases look like?

Back to Our Example

- Problem
 - Analyze my experimental data
- Requirements
 - Load CSV files
 - Calculate summary statistics
 - Identify outliers
 - Produce result file and plots

Back to Our Example (2)

● Specification

- For each CSV file
 - group measurements by experimental condition
 - compute mean, median and standard deviation
 - identify observations more than 3 standard deviations from the group mean
 - ...

● Constraints

- For the plots, use matplotlib
- Don't use Python versions older than 3.14
- Don't modify the original data files

Back to Our Example (3)

- Acceptance criteria
 - A handcrafted example (with known output) produces the same output
 - `results.csv` has to pass some statistical tests
- Examples
 - Provide typical snippets of experimental data
 - Also include some edge cases

Resolving Ambiguities

- What happens if we don't provide this information?
- LLMs always give an answer
- They resolve ambiguities by filling in the gaps
 - Based on the most plausible options from training
- This does not usually get you what you want.



Decomposition

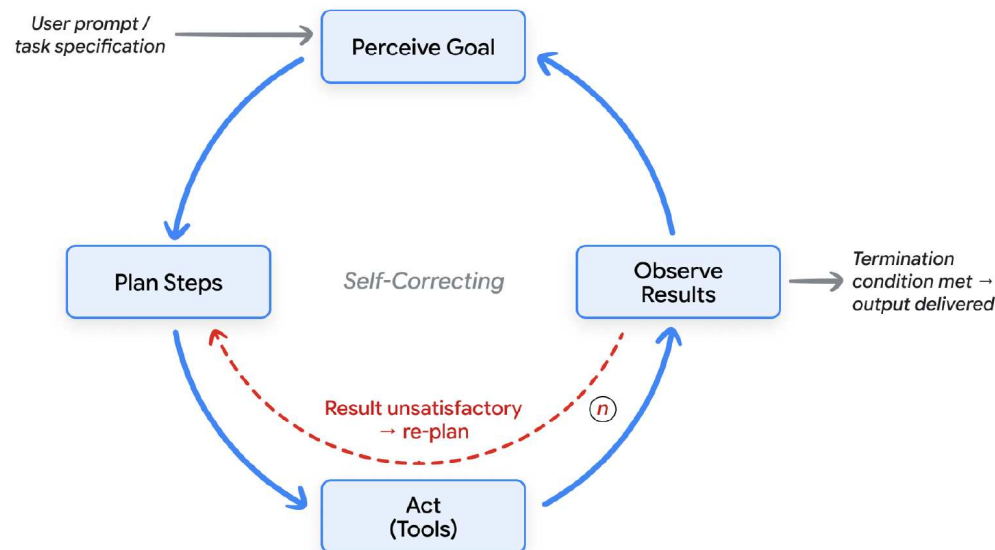
- Once the specification is done, ask the following:
Can this be broken down into smaller subtasks?
- For analyzing experimental data, this could look like
 - Load data (specifying input format + validation)
 - Group observations
 - Calculate statistics
 - Detect outliers
 - Write results
 - Generate plots
 - Decide whether to accept analysis

Decomposition (2)

- Gives you checkpoints
 - Instead of producing thousands of lines of code,
 - implements and verifies one part at a time
- Also helps with resolving ambiguities
 - Can now also be done for one part at a time
- We talk more about decomposition in the next chapter

Mapping to Agentic Loop

- Maps nicely onto the loop from the previous chapter:
 - Perceive Goal – Problem
 - Plan Steps – Specification
 - Act – Implement
 - Observe Results – Check Acceptance Criteria
- If it isn't working, refine specification



Summary

- Agentic coding is not about finding the “magic” prompt
- It's about (systematically) getting rid of ambiguities
 - If we don't, an LLM will fill gaps on its own
- So, if things don't work
 - Don't ask an agent to fix the code
 - Instead, go over the specification

Chapter 5

Structuring the Code

Motivation

- We could throw all the code into one big file
- For instance, by prompting
 - Write a program that reads data from CSV-files, cleans it, computes statistics, detects outliers, ...
 - Implement the above specification
- An LLM would happily create a single program containing hundreds of lines of code
- Program may work correctly but why is this a bad idea?

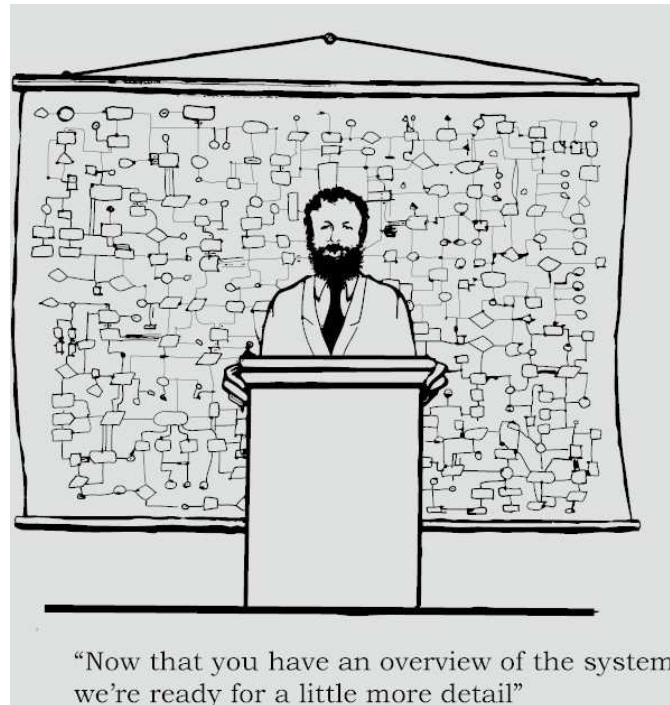
Motivation (2)



- Code like this is hard to follow and maintain
- This is the case not just for humans
- An LLM will also have problems
 - The size of the context window is limited
 - If the program becomes too large, it will not fit
 - So, an LLM may be missing important information

Decomposition

- Specification says what and how things are done
- Decomposition breaks functionality into pieces and sets up boundaries between them
 - Determines what is done where
 - Defines how pieces interact



Architecture vs. Design

- Software developers use different terminology when it comes to structuring code
 - Architecture: high-level view of a system's structure
 - Design: more concerned with individual components
- There is no strict boundary between the two
- We're not going into details here
 - Important for you: you need to structure the code

Modules

- In Python, modules structure your code
- A module is simply a file containing Python code
- E.g., we could introduce the following structure:

```
project/  
├── data/  
│   ├── loader.py  
│   └── cleaning.py  
├── analysis/  
│   ├── statistics.py  
│   └── outliers.py  
├── reporting/  
│   └── results.py  
├── visualization/  
│   └── plot.py  
└── main.py
```

Modules (2)

- It's not about how many files there should be
- In software development, modules are about
 - responsibilities
 - interfaces
 - dependencies
- In agentic coding, they also set boundaries

Responsibilities

- Ideally, a module should perform one specific task
- Questions to ask yourself are:
 - Does every module have a clear purpose?
 - Are any modules doing too many unrelated things?
 - Are there unnecessary modules?

Interfaces

- The interface defines how the outside world interacts with a module
- Should not reveal internal implementation details
- States which services a module provides
- Questions to ask yourself are:
 - What does each module expose?
 - Are the interfaces simple?
 - How much do modules need to know about each other?

Dependencies

- A module A depends on module B if it relies on the functionality of B to work correctly
- There is also indirect, transitive dependency
- High dependency makes code hard to change or test independently
 - Which modules depend on which?
 - Are there circular dependencies?
 - Could a change in one modules unnecessarily affect many others?

Modules (2)

- It's not about how many files there should be
- It's about separating responsibilities
- Let's assume everything is in `main.py`
- You want to modify the outlier detection
 - The agent will edit `main.py` and may change other parts of the code as well
- Now assume the code for the outlier detection is solely in `outlier.py`
 - You can tell the agent to only touch code in this file

Interfaces

















